ALGORITHMUS, PROGRAMM, PROZESS

- **Algorithmus:** Handlungsvorschrift, die ein Problem in **endlicher Zeit** löst.
- **Programm:** Umsetzung eines oder mehrerer Algorithmen in einer Programmiersprache; darf endlos laufen.
- **Prozess: laufendes** Programm mit Daten, Variablenwerten und Verwaltungsinformationen des Betriebssystems.
- Uhrzeit jede Sekunde anzeigen: Programm, aber kein Algorithmus (endet nie). Darstellung: Struktogramm, UML-Aktivitätsdiagramm.

MASCHINENSPRACHE → ASSEMBLER → HOCHSPRACHE

- **Maschinensprache:** binäre Befehle, je Prozessortyp anders, kaum lesbar.
- **Assembler:** kurze Namen für Befehle (ADD) und Adressen; 1 Assemblerbefehl = 1 Maschinenbefehl.
- **Hochsprache** (Python, Java, C++): für Menschen lesbar, braucht Compiler oder Interpreter.
- **Register:** Speicher direkt im Prozessor. Prinzip: laden → rechnen → zurückschreiben.

SYNTAX- UND SEMANTIKFEHLER

	Syntaxfehler	Semantikfehler
Was?	falsche Schreibweise, z. B. fehlende Klammer	richtig geschrieben, falsche Bedeutung
Folge	Compiler übersetzt nicht	läuft, falsches Ergebnis, keine Meldung

Typfalle (Java): int / int ergibt int, der Rest wird abgeschnitten.

```
double res  = 10 / 3;           // 3.0
double res2 = (double) 10 / 3;  // 3.333…
```

VARIABLEN UND DATENTYPEN (JAVA)

Variable = Speicherplatz mit Namen und Typ. Erst lesbar, wenn ein Wert zugewiesen wurde.

- Deklaration `int zaehler;` · mit Initialisierung `int summe = 0;` · Zuweisung `summe = summe + 5;`

Typ	Größe	Bereich
byte	8 Bit	−128 … 127
short	16 Bit	−32 768 … 32 767
int	32 Bit	ca. ±2,1 Mrd.
long	64 Bit	ca. $\pm 9{,}2 \cdot 10^{18}$
float / double	32 / 64 Bit	Gleitkomma (IEEE 754)
char	16 Bit	ein Unicode-Zeichen
boolean	laut Folie 1 Bit	true, false

COMPILER UND INTERPRETER

	Compiler	Interpreter
Wann?	vorab, ganzes Programm	zur Laufzeit, Befehl für Befehl
Ergebnis	Maschinenprogramm für bestimmte Hardware	direkte Ausführung

Zwischenschritt möglich: maschinenunabhängiger **Bytecode** (Java, Python).

Compilerphasen: 1. Scanner (lexikalische Analyse, Grundsymbole) → 2. Parser (syntaktische Analyse, Syntaxbaum) → 3. semantische Analyse + Optimierung → 4. Codeerzeugung + -optimierung → Maschinenprogramm

LINKER, LADER, DEBUGGER

- **Linker:** verbindet einzeln übersetzte Module zu einem lauffähigen Programm.
- **statisch:** alles in eine Datei gebunden. + alles sicher vorhanden, – große Datei.
- **dynamisch:** Funktionen erst zur Laufzeit gesucht (DLLs, Shared Libraries). + gemeinsame Speichernutzung, Bibliotheken austauschbar, – fehlt eine Funktion → Abbruch.
- **Lader:** bringt das Programm in den Arbeitsspeicher, addiert den Startwert (Offset) zu allen Adressen. Laden an 0x2000: Sprungziel 0x0040 → **0x2040**.
- **Debugger:** Programm Schritt für Schritt, zeigt Speicher, Register, Variablen. **IDE** (z. B. Eclipse): Editor + Compiler + Debugger.

REKURSION

- **Teile und Herrsche** = Prinzip: Aufgabe in gleichartige kleinere Teilaufgaben zerlegen.
- **Rekursion** = Technik: Funktion ruft sich selbst mit kleinerer Eingabe auf. Braucht eine **Abbruchbedingung**.

```
long fakrek(int n) {
    if (n == 1 || n == 0) return 1;  // Abbruch
    return n * fakrek(n - 1);        // Selbstaufruf
}
```

fakrek(4): 4 · fakrek(3) → 3 · fakrek(2) → 2 · fakrek(1) = 1 (Abbruch). Rückweg: 2 · 1 = 2 → 3 · 2 = 6 → 4 · 6 = **24**.

fakrek(5) = **120**, 5 Aufrufe. Offene Aufrufe liegen auf dem Aufrufstapel (Stack).

TYPISCHE FEHLER

- „Ein Programm muss terminieren.“ Nur der Algorithmus muss endlich sein.
- Assembler als Hochsprache bezeichnen.
- 7 / 2 in Java = 3,5 erwarten. Ergebnis 3, das ist ein Semantikfehler.
- Linker und Lader verwechseln: Linker baut zusammen, Lader lädt in den Speicher.
- Teile und Herrsche mit Rekursion gleichsetzen (Prinzip vs. Technik).
- Abbruchbedingung vergessen → endlose Aufrufe, Aufrufstapel läuft über.